```
Waiting
0
2
prog.rb:4:in `block (2 levels) in <main>': Boom! (RuntimeError)
prog.rb:4:in `block (2 levels) in <main>': Boom! (RuntimeError)
```

Controlling the Thread Scheduler

In a well-designed application, you'll normally just let threads do their thing. Building timing dependencies into a multithreaded application is generally considered to be bad form because it makes the code far more complex and also prevents the thread scheduler from optimizing the execution of your program.

That said, the Thread class provides a number of methods that control or give hints to the scheduler. Invoking stop stops the current thread, and invoking run arranges for a particular thread to be run. The method pass deschedules the current thread, allowing others to run, and join and value block the calling thread until a given thread finishes. These last two are the only low-level thread control methods that the average program should use. In fact, we believe that the low-level thread control methods are too complex and dangerous to be used correctly in programs we write. Fortunately, Ruby has support for higher-level thread synchronization.

Synchronization via Mutual Exclusion

Let's start by looking at a simple example of a race condition—multiple threads updating a shared variable:

```
sum = 0
threads = 10.times.map do
  Thread.new do
    100_000.times do
      new_value = sum + 1
      print "#{new_value}   " if new_value % 250_000 == 0
      sum = new_value
    end
  end
end
threads.each(&:join)
puts "\nsum = #{sum}"
```

produces:

```
250000   250000   250000   250000   250000   250000
sum = 349999
```

We create 10 threads, and each increments the shared sum variable 100,000 times. And yet, when the threads all finish, the final value in sum is considerably less than 1,000,000. We have a race condition. The reason is the print call that sits between the code that calculates the new value and the code that stores it back into sum. In one thread, the updated value gets calculated—let's say that the value of sum is 99,999, so new_value will be 100,000. Before storing the new value back into sum, we call print, and that causes another thread to be scheduled (because the thread blocks waiting for the I/O to complete). So a second thread also fetches the value of 99,999 and increments it. It stores 100,000 into sum, then loops around again and stores 100,001, and 100,002, and so on. Eventually the original thread continues running because it finished writing its message. It immediately stores its value of 100,000